Why Extract Skills?

In the warm-up, you installed prebuilt skills (file-processing, pptx). Now you'll build one from scratch and understand why extracting procedures from steering into skills matters.

Skills are portable instruction packages that follow the open Agent Skills standard. They bundle instructions, scripts, and templates into reusable packages that Kiro can activate when relevant to your task.

Learn more: IDE Agent Skills docs | CLI Agent Skills docs

Skills solve the context overload problem with progressive disclosure:

1. **Discovery** — At startup, Kiro loads only the `name` and `description` of each skill
2. **Activation** — When your request matches a skill's description, Kiro loads the full instructions
3. **Execution** — Kiro follows the instructions, loading scripts or reference files only as needed

By now, your steering file is doing double duty — it contains both the rules (what to do) AND the procedures (how to do it). As the file grows, every conversation pays the token cost for loading all that procedural detail, even when you're just asking a simple question.

Stays in Steering	Moves to Skill
Purpose and context	Step-by-step API query workflow
Data source priority order	Exact MCP tool calls and parameters
Report format requirements	CLI commands with examples
Urgency categories	endoflife.date API curl commands
Rules (AWS docs first, date format, etc.)	Cross-referencing logic
Template references	Error handling procedures
Success criteria	Output format specification

Why split?

- **Steering loads every conversation** (~rules, always present)
- **Skill loads only when you ask about EOL data** (~procedure, on-demand)
- **References load only when the skill needs deeper detail** (~documentation)

Step 2.1: Create the Skill Directory

Create the skill directory following the agentskills.io specification:

```
mkdir -p .kiro/skills/retrieve-eol-data/references
```

```
.kiro/skills/retrieve-eol-data/
├── SKILL.md              # Required: metadata + instructions
└── references/
    └── data-sources.md   # Optional: detailed data source documentation
```

Step 2.2: Create the SKILL.md

Create `.kiro/skills/retrieve-eol-data/SKILL.md` with the following content:

```
cat > .kiro/skills/retrieve-eol-data/SKILL.md << 'EOF'
---
name: retrieve-eol-data
description: >
  Retrieves End of Service (EOS) and End of Life (EOL) dates for AWS cloud infrastructure
  resources by querying multiple authoritative data sources. Use when asked to "check EOL dates",
  "get deprecation info", "find end of life", "check version support", "retrieve lifecycle data",
  "EOL lookup", or when generating EOL/EOS reports.
license: MIT
compatibility: >
  Requires AWS CLI configured with valid credentials. Requires network access for
  endoflife.date API and AWS Knowledge MCP server.
metadata:
  author: Operations Team
  version: "1.0.0"
  tags: aws eol eos deprecation lifecycle compliance
---

# Retrieve EOL Data

Retrieve EOL dates by querying multiple data sources in strict priority order.

## Data Source Priority
1. AWS Official Documentation (via AWS Knowledge MCP)
2. AWS Health Dashboard (account-specific)
3. AWS CLI APIs (real-time version data)
4. endoflife.date API (non-AWS upstream/community only)

## Step 1: Query AWS Official Documentation (Primary)
Use the AWS Knowledge MCP `search_documentation` and `read_documentation` tools.

## Step 2: Query AWS Health Dashboard (Account-Specific)
Check for account-specific planned lifecycle events. Must use `us-east-1` region.

## Step 3: Query AWS CLI APIs (Version Details)
Use `aws rds describe-db-engine-versions`, `aws eks describe-cluster`, `aws lambda list-functions`.

## Step 4: Query endoflife.date API (Non-AWS Only)
Use only for upstream community EOL dates. Do NOT use as primary source for AWS service dates.

## Step 5: Cross-Reference and Validate
AWS official documentation always wins when dates conflict.

## Output Format
For each resource: Resource Type, Version/Runtime, EOL Date, Days Remaining, Source, Confidence.

See [data source details](references/data-sources.md) for full documentation.
EOF
```

Key Points

- `name` must be kebab-case, match the parent directory name, max 64 characters
- `description` must describe what the skill does AND when to use it (max 1024 characters)
- Keep the body under 500 lines / 5000 tokens — move detailed docs to `references/`
- Kiro loads only the `name` and `description` at startup (~100 tokens), full instructions load only when activated

Step 2.3: Add Reference Documentation

Create `.kiro/skills/retrieve-eol-data/references/data-sources.md` with detailed documentation for each data source:

```
cat > .kiro/skills/retrieve-eol-data/references/data-sources.md << 'EOF'
# Data Sources Reference

## 1. AWS Knowledge MCP Server (Primary)
Available Tools: search_documentation, read_documentation, recommend, get_regional_availability

## 2. AWS Health Dashboard
Event type: AWS_{SERVICE}_PLANNED_LIFECYCLE_EVENT
Important: Only available in us-east-1 region. Requires Business or Enterprise Support plan.

## 3. AWS CLI APIs
- aws rds describe-db-engine-versions
- aws eks describe-cluster
- aws lambda list-functions

## 4. endoflife.date API (Non-AWS Only)
Free REST API: https://endoflife.date/api/{product}.json
Use for non-AWS products only.

## 5. AWS Official Documentation (Direct Links)
- Lambda: https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtimes.html
- RDS: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/extended-support.html
- EKS: https://docs.aws.amazon.com/eks/latest/userguide/kubernetes-versions.html
EOF
```

Step 2.4: Refactor the Steering File

Update your existing steering file to remove the procedural content and point to the skill instead. The refactored steering file should be ~60% smaller — procedures now load on-demand via the skill.

```
cat > .kiro/steering/eol-reporting.md << 'EOF'
---
inclusion: always
---
# EOS/EOL Reporting Agent

## Purpose
You are an AI agent specialized in tracking End of Service (EOS) and End of Life (EOL) dates
for cloud infrastructure resources.

## Your Responsibilities
1. Inventory AWS Resources
2. Retrieve EOL Data — Use the `retrieve-eol-data` skill for step-by-step procedures
3. Categorize by Urgency (Critical < 6 months, Warning 6-12 months, Monitor 1-2 years)
4. Generate Reports

## Rules
- AWS official documentation is always the primary source
- Format dates consistently (YYYY-MM-DD)
- If EOL date is not available, state "EOL date unknown"

## Data Source Priority Order
1. AWS Official Documentation (via Knowledge MCP) — Always use first. Authoritative source.
2. AWS Health Dashboard — Account-specific deprecation notifications.
3. AWS CLI APIs — Real-time resource inventory and version details.
4. endoflife.API — Non-AWS upstream/community EOL dates only.

## Report Format
- Summary Section with counts by category
- Critical/Warning/Monitor sections with resource details
- Recommendations with prioritized actions
- Reference Links (🔗) and CLI Commands (🖥️) for each group

## Report Templates
- Markdown: #[[file:.kiro/steering/eol-report/eol-report-template.md]]
- HTML: #[[file:.kiro/steering/eol-report/eol-report-template.html]]
EOF
```

Step 2.5: Test the Skill

Before testing, start a fresh session so Kiro picks up the new skill:

Kiro IDE | Kiro CLI

1. Start a new conversation (the skill is detected at session start)
2. Verify the skill appears in the context by checking the Skills section

Now verify the skill works with the refactored steering file:

1. Test skill auto-activation:

```
Check the EOL dates for all my Lambda functions
```

Kiro should automatically activate the `retrieve-eol-data` skill based on the description match.

2. Test with explicit EOL query:

```
What are the EOL dates for Python 3.9 and Node.js 20?
```

3. Test full report generation (steering + skill together):

```
Generate a weekly EOS/EOL report following the format defined in the steering file.
```

The steering file provides the rules and format. The skill provides the procedure for retrieving EOL data.

Progressive Disclosure in Action

Level	What Loads	Token Cost
Level 1 (Metadata)	`name` and `description` at startup	~100 tokens
Level 2 (Instructions)	Full SKILL.md body when activated	<5000 tokens
Level 3 (References)	`references/data-sources.md` only as needed	Variable

Directory Structure So Far

```
.kiro/
├── skills/
│   └── retrieve-eol-data/
│       ├── SKILL.md              # Procedural playbook (on-demand)
│       └── references/
│           └── data-sources.md   # Detailed docs (loaded as needed)
└── steering/
    ├── eol-reporting.md          # Rules & format (always loaded)
    └── eol-report/
        ├── eol-report-template.html
        ├── eol-report-template.md
        └── logo-2c2p-d.webp
```

Key Takeaway

Skills package procedural knowledge that loads on-demand, following the Agent Skills open standard. This keeps context lean while keeping all knowledge accessible. In the next phase, we'll create a dedicated agent identity.

Skill Structure
This skill uses a simple structure: `SKILL.md` + `references/`. In Section 3, you'll see a skill that also uses `scripts/` for automated checks and `assets/` for output templates - the full skill directory structure supported by the agentskills.io specification.